

Freight Rate Prediction

Machine Learning Engineer assessment — report

Mohamed Amin Makni · October 2026 · Code: [GitHub link] · Video: [Loom link]

1. Summary

- **Task:** predict `posted_rate` (the price of a truck load, in \$) for 12,000 loads in Nov–Dec 2025, using 48,000 labelled loads from Jan–Oct 2025.
- **Data quality:** 677 prices (1.4%) are corrupted, about 3.5× too low or too high; 292 weights have a flipped sign; `weight` and `market_index` have a few missing values. Each was tested, then fixed or removed.
- **Key finding:** `quote_signal` looks like the best feature (± 0.99 correlation with price per mile), but its sign flips between months, and a label-free test shows it carries no information in Nov–Dec. I excluded it.
- **Validation:** time-based only. Fold A trains on Jan–Jul and tests on August, the one month that behaves like Nov–Dec; Fold B trains on Jan–Aug and tests on Sep–Oct.
- **Model:** gradient boosting that predicts the price **per mile**, then multiplies by distance. MAE **\$40.9 (1.74%)** on Fold A and **\$42.0 (1.80%)** on Fold B, against \$175–193 ($\approx 8\%$) for a simple baseline.

2. Data and quality issues

No data dictionary was provided, so I explored every column before cleaning, and fixed each problem only after testing it. Distance was checked first, because a price can only be judged against a reliable distance.

Issue	Evidence	Decision
Corrupted prices: 677 rows (1.41%)	Price per mile below \$1.35 or above \$4.00, in the empty gaps of the distribution; about 3.5× off, with no single correction factor	Removed from training (kept aside to report the error with them)
Negative weights: 292 rows	Made positive, they have the same distribution as normal weights (Kolmogorov–Smirnov $p = 0.70$)	Sign errors → absolute value
Missing values	<code>weight</code> 300, <code>market_index</code> 374, no other column	Weight → training median; index → same-day average
Distance reliability	99.8% of loads within $\pm 12\%$ of their lane median	Trusted as the main length feature
Coordinates	One position per city; a few are approximate	Used as location hints
New cities	8 validation cities never appear in training (12.1% of validation loads)	Use coordinates and distance, not city names

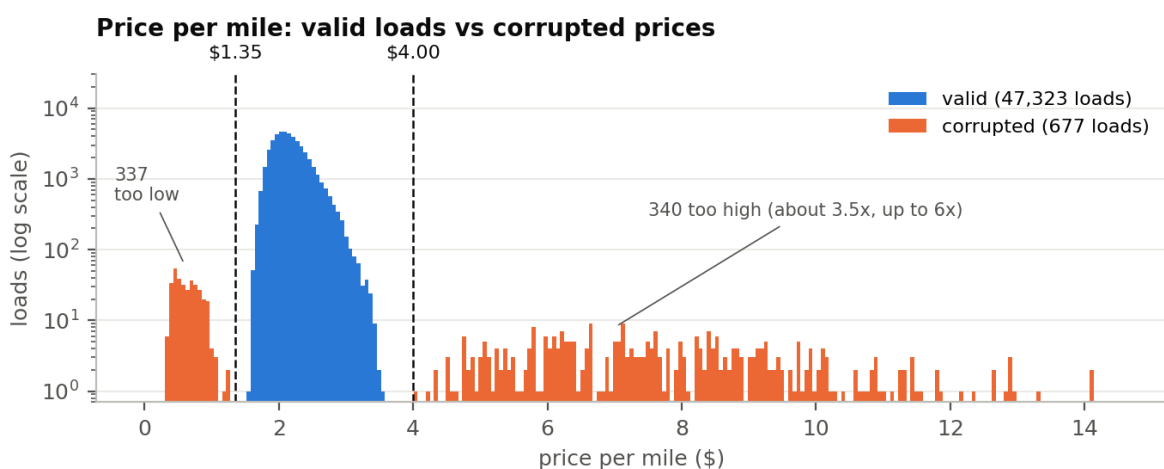


Figure 1. Price per mile. Valid loads (blue) form one block between about \$1.4 and \$3.6; the 677 corrupted prices (orange) sit apart, on both sides of the thresholds.

Main drivers of price

- **Distance** (correlation 0.91 with price) is by far the strongest driver, which is why the model works on price per mile.
- **Equipment**: Dry Van \$2.05, Flatbed \$2.22, Reefer \$2.31 per mile, at almost identical median distances.
- **Weight**: small but steady effect, +7% per mile from the lightest to the heaviest loads.
- **Date**: a weekly pattern (Thursday highest, about $\pm 1\%$) and a market level that moves from month to month.

3. Key finding: quote_signal changes behaviour

quote_signal is almost a copy of price per mile, but its direction depends on the month: it follows the price in Jan–Mar, Jun and Sep, is inverted in Apr, May, Jul and Oct, and is pure noise in August. The question is which behaviour Nov–Dec will follow, without knowing their prices.

Label-free test. If quote_signal follows price, it must also follow what drives price: Reefer loads cost more than Dry Van loads. In inverted months this gap flips; in a noise month it disappears. The test needs no prices, so it can be run on Nov–Dec. Result: in Nov–Dec the gap is zero and the correlation with distance is zero, exactly like August.

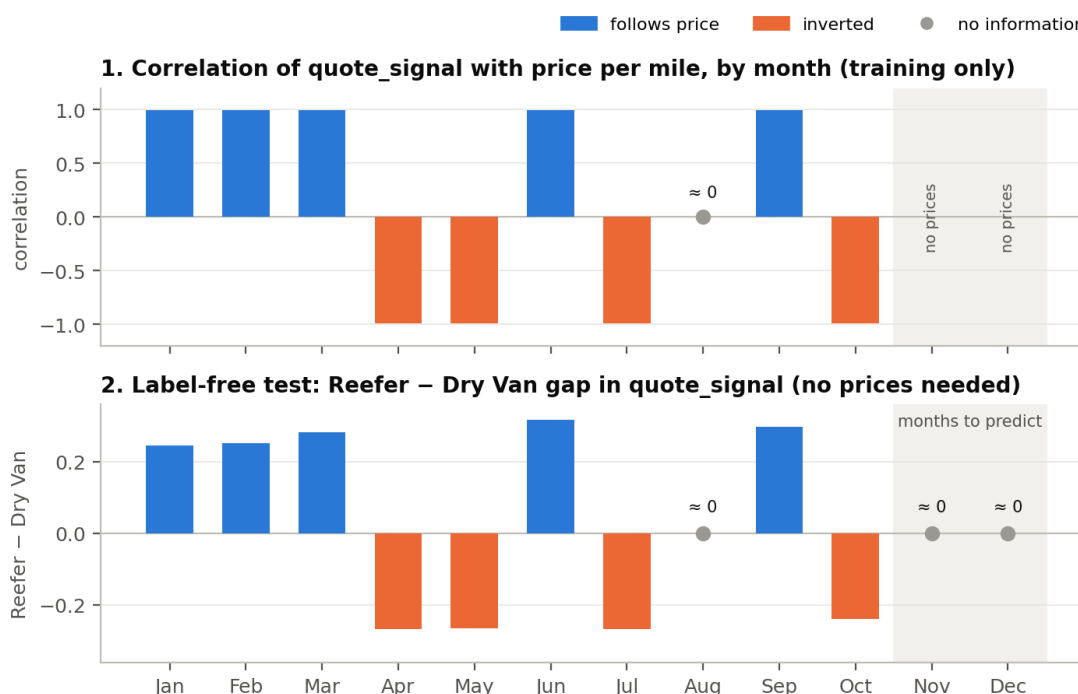


Figure 2. Top: monthly correlation of quote_signal with price per mile. Bottom: the label-free test, which needs no prices; Nov–Dec behave like August, where the signal carries no information.

A model using quote_signal wins on Sep–Oct (\$32.4) but is the worst on August (\$110.4), the month that resembles Nov–Dec, so I excluded it. market_index was also excluded: it tracks prices until July, then drifts (it falls to its lowest level in September while prices rise), and it made both folds worse.

4. Validation and split approach

The goal is to predict future months, so the split is **by time, never random**. A random split would put loads from the same days in training and test, leaking the market level and making quote_signal look excellent when it would fail on Nov–Dec.

Time-based validation: always train on the past, test on the future

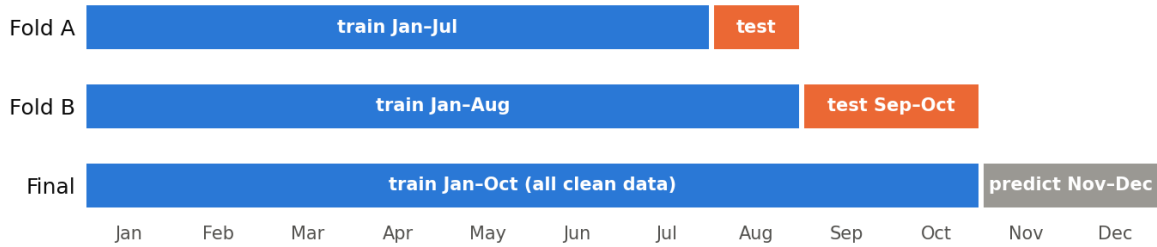


Figure 3. Each fold trains on earlier months and tests on later ones. After choosing the features, the final model is retrained on all clean Jan-Oct data.

- **Fold A: train Jan-Jul (33,253 loads) → test August (4,691).** August is the only training month whose `quote_signal` behaves like Nov-Dec, so it is the most realistic test.
- **Fold B: train Jan-Aug (37,944) → test Sep-Oct (9,379).** The most recent data, with the same two-month horizon as Nov-Dec.
- **A model has to do well on both.** Fold B alone would have chosen the `quote_signal` model; Fold A alone is a single month. Together they rule it out.
- **Metrics:** MAE in dollars (easy to read for pricing) and MAPE in percent (loads range from about \$60 to \$25,000).
- **Corrupted rows** are removed from the test folds too, so the scores measure real prices. For information, the same model on raw Sep-Oct, corrupted rows included, gives an MAE of \$98, almost all of it from the 1.4% of rows no model can predict.

5. Model and results

Model: scikit-learn HistGradientBoostingRegressor (500 trees, learning rate 0.05), a strong default for tabular data with non-linear effects, with equipment as a native categorical feature. **Features:** distance, equipment, weight, pickup and delivery coordinates, day of week.

Target: the model learns the price **per mile** and the prediction is multiplied by distance. Per-mile prices sit on one stable scale, tree models cannot extrapolate to very long trips, and this is how loads are priced in practice. It was also about \$2 more accurate than predicting the price directly on both folds.

Model	Fold A: test Aug	Fold B: test Sep-Oct
Baseline: median price per mile by equipment × distance	\$193.4 · 8.20%	\$175.1 · 8.09%
Final model (no <code>quote_signal</code>, no <code>market_index</code>)	\$40.9 · 1.74%	\$42.0 · 1.80%
Same + <code>market_index</code>	\$66.6 · 2.85%	\$116.0 · 4.83%
Same + <code>quote_signal</code>	\$110.4 · 5.33%	\$32.4 · 1.33%

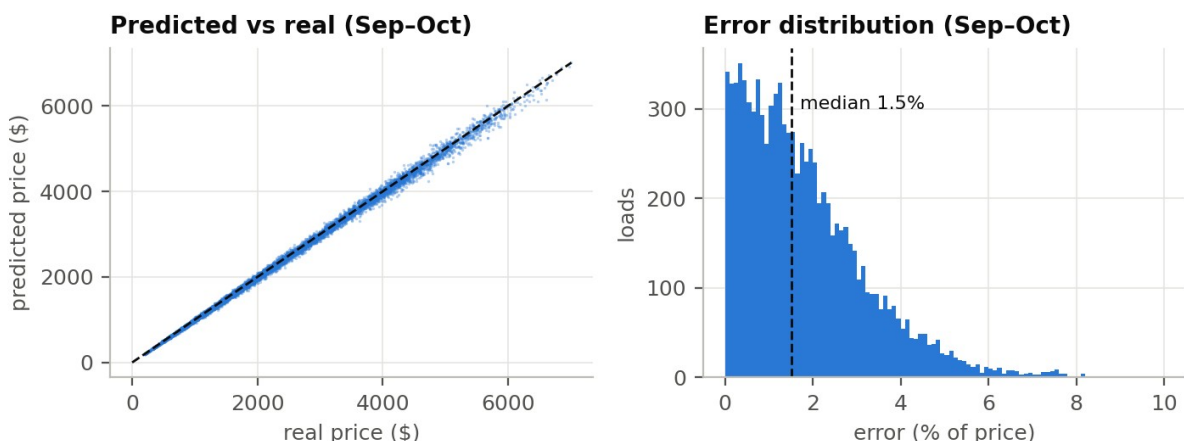


Figure 4. Final feature set on Fold B. Predictions follow the real price from \$200 to \$7,000; the median error is 1.5%.

Closer evaluation (Fold B)

- MAE \$42, median error \$28, R^2 0.998; **96.8% of loads within 5%** of the real price.
- The error in dollars grows with distance (\$11 to \$82) but stays **flat in percent (1.7-2.0%)**, as expected from per-mile modelling.
- Flatbed is the hardest equipment (2.25%), probably because it has the fewest loads.
- **Unseen cities:** with 8 cities hidden from training, the error rises only from 1.81% to 1.90%, so the model handles the 8 new validation cities through their coordinates.
- Small negative bias (-\$15): Sep-Oct prices were above the Jan-Aug average the model learned from.

6. December prediction chart

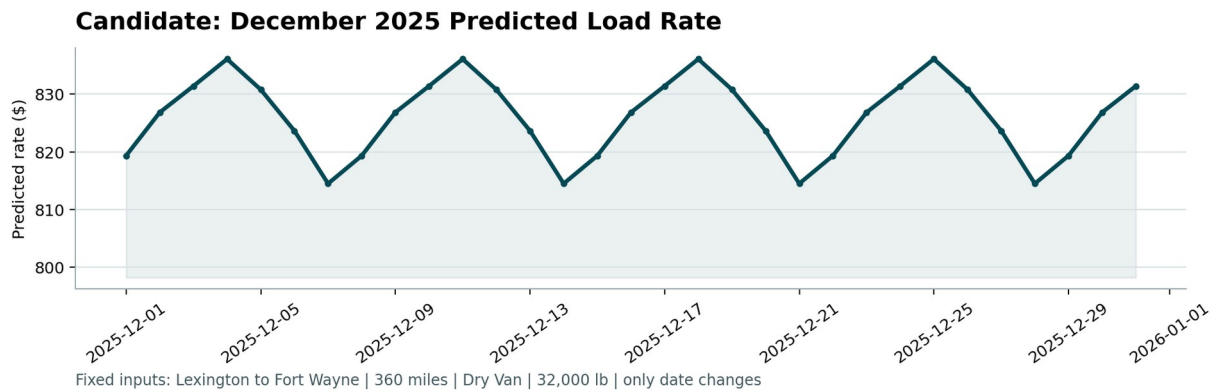


Figure 5. Produced by the provided score.py: Lexington to Fort Wayne, 360 miles, Dry Van, 32,000 lb, only the date changes.

Only the date changes, and day of week is the model's only date feature, so the curve is a stable level with a weekly pattern: highest on Thursdays (about \$836) and lowest on Sundays (about \$814), a $\pm 1.3\%$ swing that matches the training data. Holidays are not modelled, because the training data contains no December.

7. Limitations and next steps

- **Market level:** add a trend or recency weighting, or forecast the daily market level with a simple time-series model, to remove the negative bias.
- **Calendar effects:** holidays and end of quarter (a 1-2% bump appears at the end of March, June and September).
- **Monitoring:** in production, run the label-free test on `quote_signal` regularly and re-evaluate it if it becomes informative again.
- **More evaluation:** a monthly walk-forward evaluation, hyperparameter tuning with time-based folds, and prediction intervals (quantile loss) to show uncertainty.

8. How to reproduce

Install the dependencies with `pip install -r requirements.txt` (pandas, numpy, matplotlib, scikit-learn, scipy), run the notebook from top to bottom to create `validation_predictions.csv` and fill `data/deceember_chart_inputs.csv`, then run `python score.py --predictions validation_predictions.csv --december-predictions data/deceember_chart_inputs.csv`.